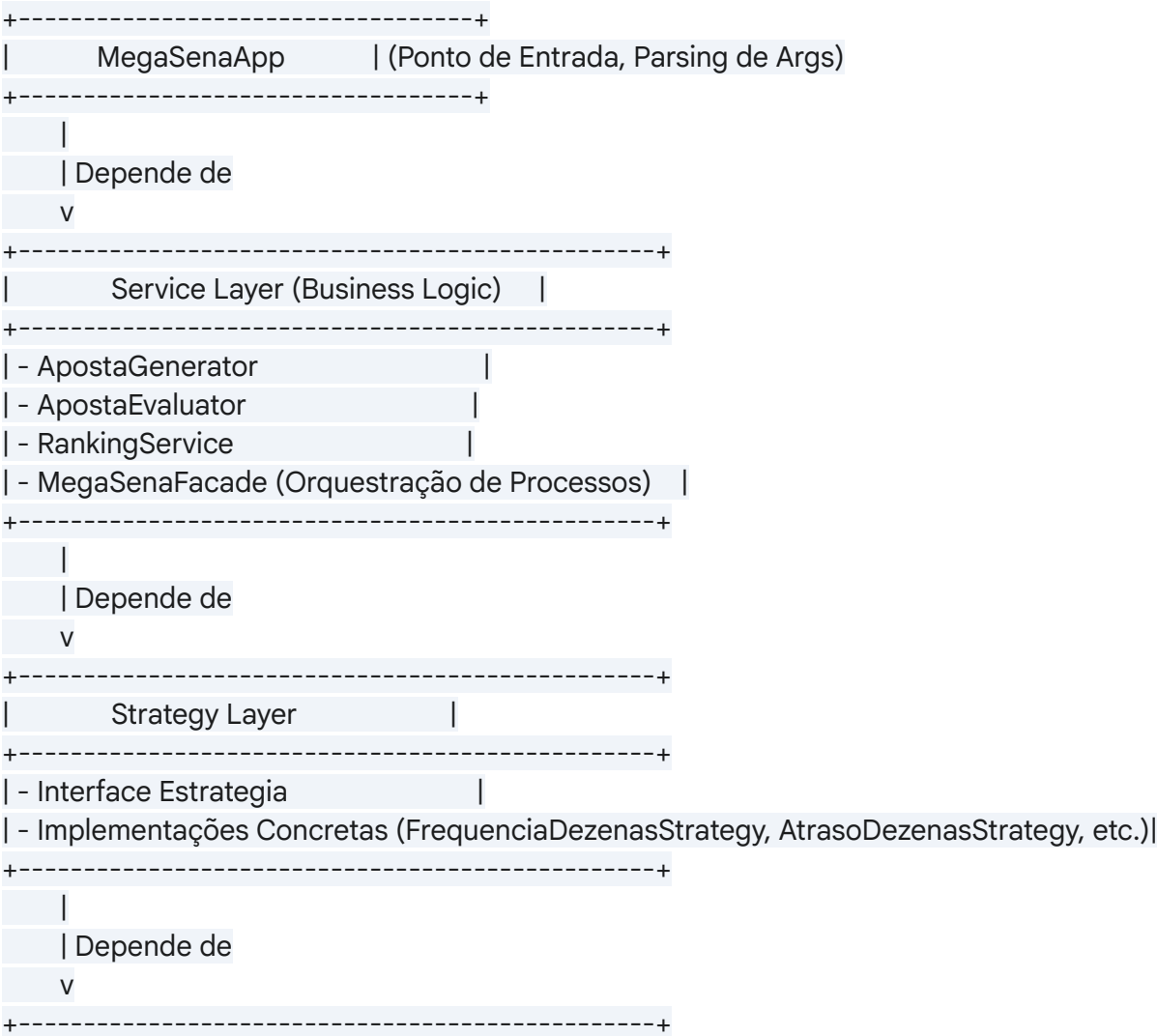


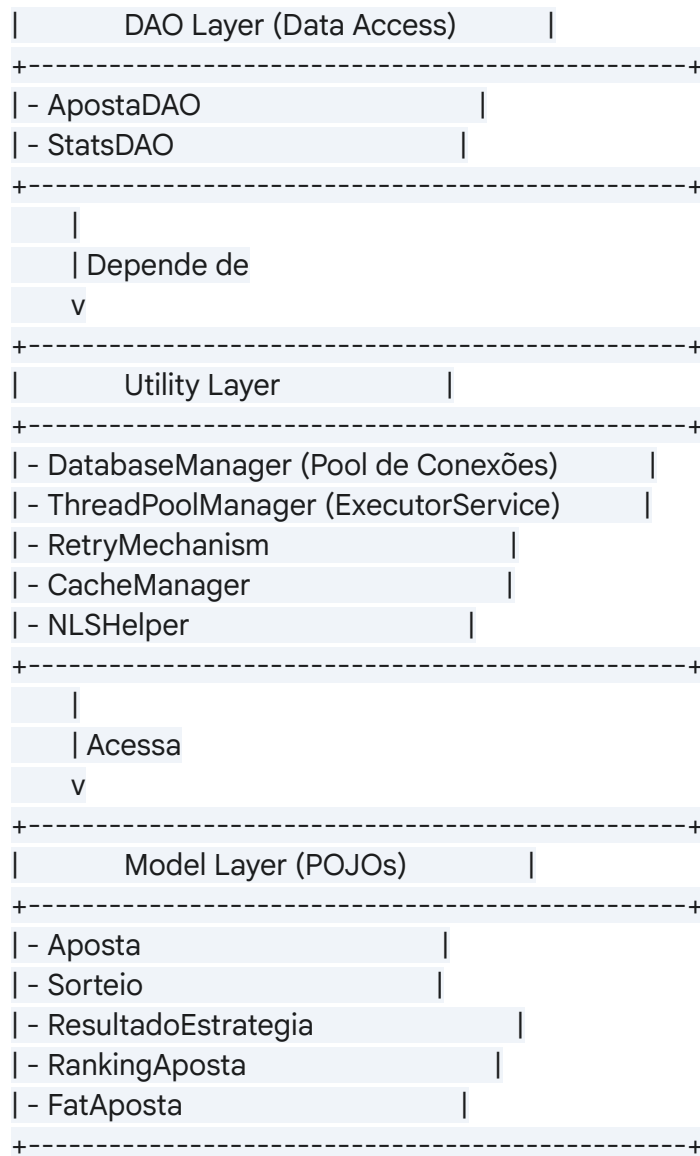
Perfeito! Com a especificação do banco de dados detalhada e um plano de desenvolvimento em fases, o próximo passo é iniciar a fase de arquitetura e design da aplicação, focando nos componentes essenciais e nas dependências.

Arquitetura da Aplicação Java: MegaSena Sugestor

1. Visão Geral da Arquitetura

A aplicação será desenvolvida em Java, seguindo um design modular e orientado a objetos para garantir manutenibilidade, escalabilidade e extensibilidade. Será utilizada uma arquitetura em camadas, facilitando a separação de responsabilidades e a implementação de recursos como paralelização e persistência.





2. Detalhamento dos Componentes

2.1. Camada de Modelo (Model Layer)

Contém as classes POJO (Plain Old Java Objects) que representam as entidades do domínio, tanto do banco de dados existente quanto das novas tabelas propostas.

- `Aposta.java`: Representa uma aposta de 6 dezenas.

Java

```
package com.megasena.model;
```

```
import java.util.Arrays;
```

```
import java.util.Set;
```

```
import java.util.TreeSet;
```

```

public class Aposta implements Comparable<Aposta> {
    private Set<Integer> dezenas;
    private double pontuacaoTotal; // Soma das pontuações das estratégias

    public Aposta(int d1, int d2, int d3, int d4, int d5, int d6) {
        this.dezenas = new TreeSet<>(Arrays.asList(d1, d2, d3, d4, d5, d6));
        if (this.dezenas.size() != 6) {
            throw new IllegalArgumentException("Uma aposta deve ter exatamente 6 dezenas únicas.");
        }
        // Validação de dezenas entre 1 e 60
        for (int dezena : this.dezenas) {
            if (dezena < 1 || dezena > 60) {
                throw new IllegalArgumentException("As dezenas devem estar entre 1 e 60.");
            }
        }
    }

    public Set<Integer> getDezenas() {
        return dezenas;
    }

    public double getPontuacaoTotal() {
        return pontuacaoTotal;
    }

    public void setPontuacaoTotal(double pontuacaoTotal) {
        this.pontuacaoTotal = pontuacaoTotal;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("[");
        int i = 0;
        for (int dezena : dezenas) {
            sb.append(String.format("%02d", dezena));
            if (i < 5) {
                sb.append(" - ");
            }
            i++;
        }
        sb.append("]");
    }
}

```

```

        sb.append(String.format("] (Pontuação: %.2f)", pontuacaoTotal));
        return sb.toString();
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Aposta aposta = (Aposta) o;
        return dezenas.equals(aposta.dezenas);
    }

    @Override
    public int hashCode() {
        return dezenas.hashCode();
    }

    @Override
    public int compareTo(Aposta other) {
        // Compara para ranking decrescente (maior pontuação primeiro)
        return Double.compare(other.pontuacaoTotal, this.pontuacaoTotal);
    }
}

```

- Sorteio.java: Representa um sorteio da Mega-Sena, correspondendo à tabela SORTEIOS.
- ResultadoEstrategia.java: Representa o resultado da aplicação de uma estratégia a uma aposta.
- RankingAposta.java: Representa uma aposta ranqueada.
- FatAposta.java: Representa um registro na tabela FAT_APOSTAS para análise de desempenho.

2.2. Camada de Acesso a Dados (DAO Layer)

Responsável pela comunicação com o banco de dados SQLite. Utilizará JDBC e o pool de conexões Apache Commons DBCP.

- ApostaDAO.java:


```

Java
package com.megasena.dao;

import com.megasena.model.Aposta;
import com.megasena.model.ResultadoEstrategia;
import com.megasena.model.RankingAposta;
import com.megasena.model.FatAposta;

```

```

import com.megasena.util.DatabaseManager;
import com.megasena.util.RetryMechanism;

import java.sql.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

public class ApostaDAO {
    private final DatabaseManager dbManager;

    public ApostaDAO(DatabaseManager dbManager) {
        this.dbManager = dbManager;
    }

    public long insertApostaGerada(Aposta aposta, String modoGeracao) throws SQLException {
        String sql = "INSERT INTO APOSTAS_GERADAS (DEZENA1, DEZENA2, DEZENA3, DEZENA4, DEZENA5, DEZENA6, DATA_GERACAO, MODO_GERACAO) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
        try (Connection conn = dbManager.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {

            List<Integer> dezenasList = new ArrayList<>(aposta.getDezenas());
            for (int i = 0; i < 6; i++) {
                pstmt.setInt(i + 1, dezenasList.get(i));
            }
            pstmt.setString(7, LocalDateTime.now().toString()); // Use ISO-8601 format
            pstmt.setString(8, modoGeracao);

            int affectedRows = RetryMechanism.executeWithRetry(() -> pstmt.executeUpdate());

            if (affectedRows == 0) {
                throw new SQLException("Falha ao inserir a aposta gerada, nenhum registro afetado.");
            }

            try (ResultSet generatedKeys = pstmt.getGeneratedKeys()) {
                if (generatedKeys.next()) {
                    return generatedKeys.getLong(1);
                } else {
                    return 0;
                }
            }
        }
    }
}

```

```

        throw new SQLException("Falha ao inserir a aposta gerada, nenhum ID obtido.");
    }
}
}
}

```

```

    public void insertResultadoEstrategia(long apostald, ResultadoEstrategia resultado) throws
SQLException {
        String sql = "INSERT INTO RESULTADO_ESTRATEGIAS (ID_APOSTA, NOME_ESTRATEGIA,
PONTUACAO, DATA_AVALIACAO) VALUES (?, ?, ?, ?)";
        try (Connection conn = dbManager.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setLong(1, apostald);
            pstmt.setString(2, resultado.getNomeEstrategia());
            pstmt.setDouble(3, resultado.getPontuacao());
            pstmt.setString(4, LocalDateTime.now().toString());
            RetryMechanism.executeWithRetry(() -> pstmt.executeUpdate());
        }
    }
}

```

```

    public void insertRankingAposta(long apostald, double somaPontuacaoTotal, int rankingFinal)
throws SQLException {
        String sql = "INSERT INTO RANKING_APOSTAS (ID_APOSTA, SOMA_PONTUACAO_TOTAL,
RANKING_FINAL, DATA_RANKING) VALUES (?, ?, ?, ?)";
        try (Connection conn = dbManager.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setLong(1, apostald);
            pstmt.setDouble(2, somaPontuacaoTotal);
            pstmt.setInt(3, rankingFinal);
            pstmt.setString(4, LocalDateTime.now().toString());
            RetryMechanism.executeWithRetry(() -> pstmt.executeUpdate());
        }
    }
}

```

```

    public void insertFatAposta(FatAposta fatAposta) throws SQLException {
        String sql = "INSERT INTO FAT_APOSTAS (ID_APOSTA, DATA_APOSTA, DATA_SORTEIO,
CONCURSO, SOMA_RANKING_ESTRATEGIAS, RESULTADO_REAL_DEZENA1,
RESULTADO_REAL_DEZENA2, RESULTADO_REAL_DEZENA3, RESULTADO_REAL_DEZENA4,
RESULTADO_REAL_DEZENA5, RESULTADO_REAL_DEZENA6, PONTUACAO_REAL,
RANKING_RESULTADO_REAL) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
        try (Connection conn = dbManager.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setLong(1, fatAposta.getIdAposta());

```

```

        pstmt.setString(2, fatAposta.getDataAposta().toString());
        pstmt.setString(3, fatAposta.getDataSorteio().toString());
        pstmt.setInt(4, fatAposta.getConcurso());
        pstmt.setDouble(5, fatAposta.getSomaRankingEstrategias());
        List<Integer> dezenasReais = new
ArrayList<>(fatAposta.getResultadoRealDezenas());
        for (int i = 0; i < 6; i++) {
            pstmt.setInt(i + 6, dezenasReais.get(i));
        }
        pstmt.setDouble(12, fatAposta.getPontuacaoReal());
        pstmt.setInt(13, fatAposta.getRankingResultadoReal());
        RetryMechanism.executeWithRetry(() -> pstmt.executeUpdate());
    }
}

```

```

    public List<Aposta> getApostasGeradas(int limit) throws SQLException {
        List<Aposta> apostas = new ArrayList<>();
        String sql = "SELECT DEZENA1, DEZENA2, DEZENA3, DEZENA4, DEZENA5, DEZENA6 FROM
APOSTAS_GERADAS LIMIT ?";
        try (Connection conn = dbManager.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setInt(1, limit);
            try (ResultSet rs = RetryMechanism.executeWithRetry(() -> pstmt.executeQuery()))
            {
                while (rs.next()) {
                    apostas.add(new Aposta(
                        rs.getInt("DEZENA1"),
                        rs.getInt("DEZENA2"),
                        rs.getInt("DEZENA3"),
                        rs.getInt("DEZENA4"),
                        rs.getInt("DEZENA5"),
                        rs.getInt("DEZENA6")
                    ));
                }
            }
        }
        return apostas;
    }
}

```

```

    public void createTables() throws SQLException {
        // SQL para criar as tabelas se não existirem
        String createApostasGeradas = "CREATE TABLE IF NOT EXISTS APOSTAS_GERADAS (" +
            "ID_APOSTA INTEGER PRIMARY KEY AUTOINCREMENT," +

```

```
"DEZENA1 INTEGER NOT NULL," +  
"DEZENA2 INTEGER NOT NULL," +  
"DEZENA3 INTEGER NOT NULL," +  
"DEZENA4 INTEGER NOT NULL," +  
"DEZENA5 INTEGER NOT NULL," +  
"DEZENA6 INTEGER NOT NULL," +  
"DATA_GERACAO TEXT NOT NULL," +  
"MODO_GERACAO TEXT NOT NULL" +  
");";
```

```
String createResultadoEstrategias = "CREATE TABLE IF NOT EXISTS  
RESULTADO_ESTRATEGIAS (" +  
"ID_RESULTADO INTEGER PRIMARY KEY AUTOINCREMENT," +  
"ID_APOSTA INTEGER NOT NULL," +  
"NOME_ESTRATEGIA TEXT NOT NULL," +  
"PONTUACAO REAL NOT NULL," +  
"DATA_AVALIACAO TEXT NOT NULL," +  
"FOREIGN KEY (ID_APOSTA) REFERENCES  
APOSTAS_GERADAS(ID_APOSTA)" +  
");";
```

```
String createRankingApostas = "CREATE TABLE IF NOT EXISTS RANKING_APOSTAS (" +  
"ID_RANKING INTEGER PRIMARY KEY AUTOINCREMENT," +  
"ID_APOSTA INTEGER NOT NULL," +  
"SOMA_PONTUACAO_TOTAL REAL NOT NULL," +  
"RANKING_FINAL INTEGER NOT NULL," +  
"DATA_RANKING TEXT NOT NULL," +  
"FOREIGN KEY (ID_APOSTA) REFERENCES  
APOSTAS_GERADAS(ID_APOSTA)" +  
");";
```

```
String createFatApostas = "CREATE TABLE IF NOT EXISTS FAT_APOSTAS (" +  
"ID_FAT_APOSTA INTEGER PRIMARY KEY AUTOINCREMENT," +  
"ID_APOSTA INTEGER NOT NULL," +  
"DATA_APOSTA TEXT NOT NULL," +  
"DATA_SORTEIO TEXT NOT NULL," +  
"CONCURSO INTEGER NOT NULL," +  
"SOMA_RANKING_ESTRATEGIAS REAL NOT NULL," +  
"RESULTADO_REAL_DEZENA1 INTEGER NOT NULL," +  
"RESULTADO_REAL_DEZENA2 INTEGER NOT NULL," +  
"RESULTADO_REAL_DEZENA3 INTEGER NOT NULL," +  
"RESULTADO_REAL_DEZENA4 INTEGER NOT NULL," +  
"RESULTADO_REAL_DEZENA5 INTEGER NOT NULL," +
```



```

        "RESULTADO_REAL_DEZENA6 INTEGER NOT NULL," +
        "PONTUACAO_REAL REAL NOT NULL," +
        "RANKING_RESULTADO_REAL INTEGER NOT NULL," +
        "FOREIGN KEY (ID_APOSTA) REFERENCES APOSTAS_GERADAS(ID_APOSTA)"
    +
    ");";

    try (Connection conn = dbManager.getConnection(); Statement stmt =
conn.createStatement()) {
        RetryMechanism.executeWithRetry(() -> stmt.execute(createApostasGeradas));
        RetryMechanism.executeWithRetry(() ->
stmt.execute(createResultadoEstrategias));
        RetryMechanism.executeWithRetry(() -> stmt.execute(createRankingApostas));
        RetryMechanism.executeWithRetry(() -> stmt.execute(createFatApostas));
    }
}
}

```

- StatsDAO.java: Para acesso de leitura às tabelas DEZ_OCORRENCIAS, AGREG_DIS_DEZ_POSICAO, SORTEIOS, ESTATISTICAS_GERAIS_DEZENAS, EST_MUD_ESTADO. Conterá com métodos para buscar frequências de dezenas, dezenas atrasadas, resultados de sorteios, etc.

2.3. Camada de Estratégias (Strategy Layer)

Define e implementa as diversas estratégias estatísticas para pontuar as apostas.

- Estrategia.java (Interface):

```

Java
package com.megasena.strategy;

import com.megasena.model.Aposta;

public interface Estrategia {
    String getNome();
    double pontuar(Aposta aposta);
}

```
- FrequenciaDezenasStrategy.java:

```

Java
package com.megasena.strategy;

import com.megasena.dao.StatsDAO;
import com.megasena.model.Aposta;

```

```

import com.megasena.util.CacheManager;

import java.sql.SQLException;
import java.util.HashMap;
import java.util.Map;

public class FrequenciaDezenasStrategy implements Estrategia {
    private final StatsDAO statsDAO;
    private final CacheManager cacheManager;
    private Map<Integer, Integer> frequenciaDezenas; // Cache local para frequências

    public FrequenciaDezenasStrategy(StatsDAO statsDAO, CacheManager cacheManager) {
        this.statsDAO = statsDAO;
        this.cacheManager = cacheManager;
    }

    @Override
    public String getNome() {
        return "FrequenciaDezenas";
    }

    @Override
    public double pontuar(Aposta aposta) {
        double pontuacao = 0;
        try {
            // Tenta obter do cache primeiro
            frequenciaDezenas = cacheManager.get("frequenciaDezenas", () -> {
                try {
                    return statsDAO.getFrequenciaTodasDezenas(); // Carrega do DB se não estiver
no cache
                } catch (SQLException e) {
                    System.err.println("Erro ao carregar frequência das dezenas do DB: " +
e.getMessage());
                    return new HashMap<>(); // Retorna vazio em caso de erro
                }
            }, 60 * 1000); // Validade de 1 minuto

            if (frequenciaDezenas.isEmpty()) {
                System.err.println("Cache de frequência de dezenas vazio ou erro ao carregar.");
                return 0; // Não pontua se não há dados de frequência
            }

            // A pontuação pode ser a soma das frequências das dezenas na aposta

```

```

        for (int dezena : aposta.getDezenas()) {
            pontuacao += frequenciaDezenas.getDefault(dezena, 0);
        }

    } catch (Exception e) {
        System.err.println("Erro na estratégia de Frequência de Dezenas: " + e.getMessage());
        // Logar a exceção adequadamente
    }
    return pontuacao;
}
}

```

- AtrasoDezenasStrategy.java: Estratégia baseada na quantidade de concursos que uma dezena não aparece.
- Outras estratégias conforme definido.

2.4. Camada de Serviço (Service Layer)

Orquestra a lógica de negócio e coordena as interações entre as outras camadas.

- ApostaGenerator.java:

```

Java
package com.megasena.service;

import com.megasena.model.Aposta;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Random;
import java.util.Set;
import java.util.stream.Collectors;
import java.util.stream.LongStream;

public class ApostaGenerator {
    private static final int MIN_DEZENA = 1;
    private static final int MAX_DEZENA = 60;
    private static final int NUM_DEZENAS_APOSTA = 6;
    private static final long TOTAL_COMBINACOES = 50_063_860L; //

    private final Random random = new Random();

    public List<Aposta> generateRandomApostas(int count) {

```

```

List<Aposta> apostas = new ArrayList<>();
for (int i = 0; i < count; i++) {
    Set<Integer> dezenas = new HashSet<>();
    while (dezenas.size() < NUM_DEZENAS_APOSTA) {
        dezenas.add(random.nextInt(MAX_DEZENA) + MIN_DEZENA);
    }
    List<Integer> sortedDezenas =
dezenas.stream().sorted().collect(Collectors.toList());
    apostas.add(new Aposta(sortedDezenas.get(0), sortedDezenas.get(1),
sortedDezenas.get(2),
sortedDezenas.get(3), sortedDezenas.get(4), sortedDezenas.get(5)));
}
return apostas;
}

```

```

public Aposta generateLexicoAposta(long index) {
    if (index < 1 || index > TOTAL_COMBINACOES) {
        throw new IllegalArgumentException("Índice de combinação lexicográfica fora do
intervalo válido [1, " + TOTAL_COMBINACOES + "]");
    }
    // Algoritmo para gerar a aposta com base no índice lexicográfico
    // Este é um algoritmo complexo, que envolve combinações e cálculos fatoriais/binomial.
    // Para simplificar o exemplo, vamos apenas esboçar a lógica.
    // A implementação completa exigiria a função 'combinationKth' ou similar.
    List<Integer> dezenas = new ArrayList<>();
    long k = NUM_DEZENAS_APOSTA;
    long n = MAX_DEZENA;
    long remainingIndex = index - 1; // Ajuste para índice base 0

    for (int i = 1; i <= NUM_DEZENAS_APOSTA; i++) {
        int currentDezena = MIN_DEZENA;
        while (true) {
            long combinationsLeft = combinations(MAX_DEZENA - currentDezena,
(int)(NUM_DEZENAS_APOSTA - i));
            if (remainingIndex < combinationsLeft) {
                dezenas.add(currentDezena);
                break;
            } else {
                remainingIndex -= combinationsLeft;
                currentDezena++;
            }
        }
    }
}

```

```

    Collections.sort(dezenas);
    return new Aposta(dezenas.get(0), dezenas.get(1), dezenas.get(2),
        dezenas.get(3), dezenas.get(4), dezenas.get(5));
}

```

```

// Função auxiliar para calcular combinações C(n, k)
private long combinations(long n, int k) {
    if (k < 0 || k > n) {
        return 0;
    }
    if (k == 0 || k == n) {
        return 1;
    }
    if (k > n / 2) {
        k = (int) (n - k);
    }
    long res = 1;
    for (int i = 1; i <= k; ++i) {
        res = res * (n - i + 1) / i;
    }
    return res;
}

```

```

public List<Aposta> generateFullCombinations() {
    // Este método geraria todas as 50.063.860 combinações, o que é inviável em memória para
    // retornar como List.
    // No modo FULL, as apostas seriam geradas e avaliadas em "streaming" ou batch,
    // e as N melhores seriam persistidas/exibidas.
    System.out.println("Modo FULL: Gerando e avaliando todas as 50.063.860 combinações.
    Isso será feito em streaming.");
    // A implementação real do modo FULL envolveria um iterador que gera as combinações sob
    // demanda
    // e as passa para o ApostaEvaluator.
    return new ArrayList<>(); // Retorna lista vazia, pois o processamento será em streaming
}

```

```

public List<Aposta> readApostasFromFile(String filePath) throws Exception {
    // Implementação para ler apostas de um arquivo TXT
    // Retornaria List<Aposta>
    return new ArrayList<>(); // placeholder
}
}

```

- `ApostaEvaluator.java`: Aplica a cadeia de estratégias a uma aposta.
- `RankingService.java`: Gerencia o ranking das apostas.
- `MegaSenaFacade.java`: Orquestra o fluxo principal da aplicação, interagindo com o `ApostaGenerator`, `ApostaEvaluator`, `RankingService` e as DAOs.

2.5. Camada de Utilitários (Utility Layer)

Contém classes de suporte e infraestrutura.

- `DatabaseManager.java`: Gerencia o pool de conexões JDBC com Apache Commons DBCP.

Java

```
package com.megasena.util;
```

```
import org.apache.commons.dbcp2.BasicDataSource;
```

```
import java.sql.Connection;
```

```
import java.sql.SQLException;
```

```
import java.util.Objects;
```

```
import java.util.concurrent.TimeUnit; // Para timeout
```

```
public class DatabaseManager {
```

```
    private static BasicDataSource dataSource;
```

```
    private final String dbUrl;
```

```
    public DatabaseManager(String dbPath) {
```

```
        Objects.requireNonNull(dbPath, "O caminho do banco de dados não pode ser nulo.");
```

```
        this.dbUrl = "jdbc:sqlite:" + dbPath;
```

```
        initializeDataSource();
```

```
    }
```

```
    private void initializeDataSource() {
```

```
        if (dataSource == null) {
```

```
            dataSource = new BasicDataSource();
```

```
            dataSource.setDriverClassName("org.sqlite.JDBC");
```

```
            dataSource.setUrl(dbUrl);
```

```
            dataSource.setMinIdle(5); // Mínimo de conexões ociosas
```

```
            dataSource.setMaxIdle(10); // Máximo de conexões ociosas
```

```
            dataSource.setMaxTotal(20); // Máximo total de conexões ativas
```

```
            dataSource.setMaxWaitMillis(TimeUnit.SECONDS.toMillis(5)); // Timeout de 5
```

```
segundos para obter conexão
```

```
            // Configurações para evitar "host busy"
```

```
            dataSource.setValidationQuery("PRAGMA integrity_check;"); // Query para validar
```

conexão

```
dataSource.setTestOnBorrow(true); // Testa a conexão ao pegá-la do pool
dataSource.setTestOnReturn(true); // Testa a conexão ao retorná-la ao pool
dataSource.setTestWhileIdle(true); // Testa conexões ociosas
dataSource.setTimeBetweenEvictionRunsMillis(TimeUnit.MINUTES.toMillis(1)); //
Intervalo de 1 min para remover ociosas
dataSource.setMinEvictableIdleTimeMillis(TimeUnit.MINUTES.toMillis(5)); // Tempo
mínimo que uma conexão pode ficar ociosa
    }
}
```

```
public Connection getConnection() throws SQLException {
    return dataSource.getConnection();
}
```

```
public void closeDataSource() {
    if (dataSource != null) {
        try {
            dataSource.close();
            System.out.println("Pool de conexões do banco de dados fechado.");
        } catch (SQLException e) {
            System.err.println("Erro ao fechar o pool de conexões do banco de dados: " +
e.getMessage());
        }
    }
}
```

- ThreadPoolManager.java: Encapsula e gerencia um ExecutorService para paralelização.

Java

```
package com.megasena.util;
```

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
```

```
public class ThreadPoolManager {
    private ExecutorService executorService;
    private final int numThreads;
```

```
    public ThreadPoolManager(int numThreads) {
        this.numThreads = numThreads > 0 ? numThreads :
Runtime.getRuntime().availableProcessors(); //
```

```

    System.out.println("Número de threads configurado: " + this.numThreads);
    this.executorService = Executors.newFixedThreadPool(this.numThreads);
}

public ExecutorService getExecutorService() {
    return executorService;
}

public void shutdown() {
    executorService.shutdown(); // Inicia o desligamento gracioso
}

public void awaitTermination(long timeout, TimeUnit unit) {
    try {
        if (!executorService.awaitTermination(timeout, unit)) { // Espera as tarefas terminarem
            executorService.shutdownNow(); // Força o desligamento se o tempo esgotar
            if (!executorService.awaitTermination(timeout, unit)) {
                System.err.println("Pool de threads não terminou.");
            }
        }
    } catch (InterruptedException ie) {
        executorService.shutdownNow();
        Thread.currentThread().interrupt(); // Restaura o status de interrupção
    }
}
}

```

- RetryMechanism.java: Implementa o sistema de retry com backoff exponencial.

Java

```

package com.megasena.util;

import java.sql.SQLException;
import java.util.concurrent.Callable;

public class RetryMechanism {
    private static final int MAX_RETRIES = 3; //
    private static final long INITIAL_BACKOFF_MILLIS = 100; // ms

    public static <T> T executeWithRetry(Callable<T> callable) throws SQLException {
        long backoff = INITIAL_BACKOFF_MILLIS;
        for (int i = 0; i < MAX_RETRIES; i++) {
            try {
                return callable.call();
            }
        }
    }
}

```



```

    } catch (SQLException e) {
        // Tratar SQLITE_BUSY ou outros erros específicos de concorrência
        if (e.getMessage().contains("SQLITE_BUSY") || e.getSQLState().equals("57P01")) {
// Exemplo de SQLSTATE para deadlock/busy
            System.err.println("Database busy or connection error, retrying... Attempt " + (i + 1)
+ "/" + MAX_RETRIES);
            if (i < MAX_RETRIES - 1) {
                try {
                    Thread.sleep(backoff);
                    backoff *= 2; // Backoff exponencial
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();
                    throw new SQLException("Retry interrupted", ie);
                }
            } else {
                System.err.println("Max retries reached. Failing over.");
                throw e; // Lança a exceção após o último retry
            }
        } else {
            throw e; // Lança outras exceções SQL imediatamente
        }
    } catch (Exception e) {
        throw new SQLException("Erro inesperado durante a operação com retry.", e);
    }
}

throw new SQLException("Falha inesperada após retries."); // Não deveria ser alcançado
}

// Sobrecarga para métodos que não retornam valor
public static void executeWithRetry(Runnable runnable) throws SQLException {
    executeWithRetry(() -> {
        runnable.run();
        return null; // Void
    });
}
}

```

- CacheManager.java: Armazenamento local de scores calculados com validade.

```

Java
package com.megasena.util;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

```

```

import java.util.concurrent.Callable;

public class CacheManager {
    private final Map<String, CacheEntry> cache = new ConcurrentHashMap<>();

    private static class CacheEntry {
        final Object value;
        final long expirationTime;

        CacheEntry(Object value, long ttlMillis) {
            this.value = value;
            this.expirationTime = System.currentTimeMillis() + ttlMillis;
        }

        boolean isExpired() {
            return System.currentTimeMillis() > expirationTime;
        }
    }

    @SuppressWarnings("unchecked")
    public <T> T get(String key, Callable<T> dataLoader, long ttlMillis) throws Exception {
        CacheEntry entry = cache.get(key);
        if (entry == null || entry.isExpired()) {
            System.out.println("Cache miss for key: " + key + ". Loading data...");
            T value = dataLoader.call();
            cache.put(key, new CacheEntry(value, ttlMillis));
            return value;
        }
        System.out.println("Cache hit for key: " + key);
        return (T) entry.value;
    }

    public void invalidate(String key) {
        cache.remove(key);
    }

    public void invalidateAll() {
        cache.clear();
    }
}

```

- NLSHelper.java: Utilitário para formatação de mensagens, datas e valores conforme a

NLS do ambiente ou PT-BR.

2.6. Ponto de Entrada da Aplicação

- MegaSenaApp.java:

```
Java
package com.megasena.app;

import com.megasena.service.MegaSenaFacade;
import com.megasena.util.DatabaseManager;
import com.megasena.util.ThreadPoolManager;
import com.megasena.util.NLSHelper;

import java.io.IOException;
import java.sql.SQLException;
import java.util.Locale;

public class MegaSenaApp {

    public static void main(String[] args) {
        // NLS - Detecção automática ou Português Brasil
        NLSHelper.setLocale(Locale.getDefault()); // Tenta detectar NLS do ambiente
        if (NLSHelper.getLocale().getLanguage().equals(new Locale("pt",
"BR").getLanguage())) {
            System.out.println(NLSHelper.getMessage("app.starting.pt_BR"));
        } else {
            System.out.println(NLSHelper.getMessage("app.starting.en_US"));
        }
    }

    // Parâmetros POSIX
    int numApostas = 0;
    String modoGeracao = null;
    int numThreads = Runtime.getRuntime().availableProcessors(); // Default: número de
núcleos
    String dbPath = "MegaPower.db"; // Default
    int verbosityLevel = 1; // Default

    // Parsing dos argumentos de linha de comando
    try {
        for (int i = 0; i < args.length; i++) {
            switch (args[i]) {
                case "-n":
                    if (i + 1 < args.length) numApostas = Integer.parseInt(args[++i]);
            }
        }
    }
```

```

        break;
    case "-m":
        if (i + 1 < args.length) modoGeracao = args[++i].toUpperCase();
        break;
    case "-t":
        if (i + 1 < args.length) numThreads = Integer.parseInt(args[++i]);
        break;
    case "-d":
        if (i + 1 < args.length) dbPath = args[++i];
        break;
    case "-v":
        if (i + 1 < args.length) verbosityLevel = Integer.parseInt(args[++i]);
        break;
    default:
        System.out.println(NLSHelper.getMessage("app.unknown_param") + args[i]);
        printUsage();
        return;
    }
}
} catch (NumberFormatException e) {
    System.err.println(NLSHelper.getMessage("app.invalid_number_format") +
e.getMessage());
    printUsage();
    return;
} catch (IllegalArgumentException e) {
    System.err.println(NLSHelper.getMessage("app.invalid_param_value") +
e.getMessage());
    printUsage();
    return;
}
}

```

```

// Validação dos parâmetros
if (modoGeracao == null || (!modoGeracao.equals("RANDOM") &&
!modoGeracao.equals("LEXICO") && !modoGeracao.equals("FULL") &&
!modoGeracao.equals("FILE"))) {
    System.err.println(NLSHelper.getMessage("app.missing_mode"));
    printUsage();
    return;
}

```

```

if (!modoGeracao.equals("FULL") && numApostas <= 0) { // -n é obrigatório exceto no
modo FULL
    System.err.println(NLSHelper.getMessage("app.missing_num_apostas"));
}

```

```

        printUsage();
        return;
    }

    if (numThreads <= 0) {
        System.err.println(NLSHelper.getMessage("app.invalid_num_threads"));
        numThreads = Runtime.getRuntime().availableProcessors();
        System.err.println(NLSHelper.getMessage("app.using_default_threads") +
numThreads);
    }

    DatabaseManager dbManager = null;
    ThreadPoolManager threadPoolManager = null;
    try {
        dbManager = new DatabaseManager(dbPath);
        // Criar as tabelas se não existirem
        new com.megasena.dao.ApostaDAO(dbManager).createTables();

        threadPoolManager = new ThreadPoolManager(numThreads);
        MegaSenaFacade facade = new MegaSenaFacade(dbManager,
threadPoolManager, verbosityLevel);

        long startTime = System.currentTimeMillis();

        switch (modoGeracao) {
            case "RANDOM":
                facade.generateAndEvaluateRandomApostas(numApostas);
                break;
            case "LEXICO":
                facade.generateAndEvaluateLexicoApostas(numApostas); // numApostas aqui é
o índice inicial
                break;
            case "FULL":
                facade.evaluateAllCombinations(numApostas); // numApostas aqui é o N de
maior ranking
                break;
            case "FILE":
                facade.evaluateApostasFromFile("apostas.txt"); // Nome do arquivo fixo ou
passado como outro param
                break;
        }

        long endTime = System.currentTimeMillis();

```

```

        System.out.println(NLSHelper.getMessage("app.execution_time") + (endTime -
startTime) + NLSHelper.getMessage("app.milliseconds"));

    } catch (SQLException e) {
        System.err.println(NLSHelper.getMessage("app.db_error") + e.getMessage());
        e.printStackTrace();
    } catch (IOException e) {
        System.err.println(NLSHelper.getMessage("app.file_error") + e.getMessage());
        e.printStackTrace();
    } catch (Exception e) {
        System.err.println(NLSHelper.getMessage("app.general_error") + e.getMessage());
        e.printStackTrace();
    } finally {
        if (threadPoolManager != null) {
            threadPoolManager.shutdown();
            threadPoolManager.awaitTermination(60, TimeUnit.SECONDS); // Espera 60s pelo
desligamento gracioso
        }
        if (dbManager != null) {
            dbManager.closeDataSource(); // Fechar o pool de conexões
        }
    }
}

private static void printUsage() {
    System.out.println(NLSHelper.getMessage("app.usage_header"));
    System.out.println(NLSHelper.getMessage("app.param_n"));
    System.out.println(NLSHelper.getMessage("app.param_m"));
    System.out.println(NLSHelper.getMessage("app.param_t"));
    System.out.println(NLSHelper.getMessage("app.param_d"));
    System.out.println(NLSHelper.getMessage("app.param_v"));
}
}

```

- **NLSHelper (exemplo de implementação simplificada):**

Java

```
package com.megasena.util;
```

```
import java.text.NumberFormat;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Locale;
import java.util.ResourceBundle;
```

```

public class NLSHelper {
    private static Locale currentLocale = new Locale("pt", "BR"); // Default to Brazilian
    Portuguese
    private static ResourceBundle messages;

    static {
        loadMessages();
    }

    public static void setLocale(Locale locale) {
        if (locale != null) {
            currentLocale = locale;
            loadMessages();
        }
    }

    public static Locale getLocale() {
        return currentLocale;
    }

    private static void loadMessages() {
        try {
            messages = ResourceBundle.getBundle("messages", currentLocale);
        } catch (Exception e) {
            System.err.println("Could not load resource bundle for locale " + currentLocale + ".
            Falling back to en_US.");
            currentLocale = new Locale("en", "US"); // Fallback
            messages = ResourceBundle.getBundle("messages", currentLocale);
        }
    }

    public static String getMessage(String key) {
        try {
            return messages.getString(key);
        } catch (Exception e) {
            System.err.println("Missing message key: " + key + " for locale " + currentLocale);
            return "???" + key + "???"; // Fallback for missing keys
        }
    }

    public static String formatDateTime(LocalDateTime dateTime) {
        return
    
```

```
dateTime.format(DateTimeFormatter.ofPattern(getMessage("format.datetime"),
currentLocale));
}
```

```
public static String formatCurrency(double value) {
    NumberFormat currencyFormat =
NumberFormat.getCurrencyInstance(currentLocale);
    return currencyFormat.format(value);
}
```

```
public static String formatNumber(double value) {
    NumberFormat numberFormat =
NumberFormat.getNumberInstance(currentLocale);
    return numberFormat.format(value);
}
}
```

■ **Exemplo de arquivo messages.properties (no src/main/resources):**

Properties

messages.properties (Default - Inglês)

app.starting.en_US=MegaSena Suggester Application Starting...

app.starting.pt_BR=Iniciando Aplicação Sugeridora Mega-Sena...

app.unknown_param=Unknown parameter:

app.invalid_number_format=Invalid number format for parameter:

app.invalid_param_value=Invalid parameter value:

app.missing_mode=Generation mode (-m) is mandatory.

app.missing_num_apostas=Number of bets (-n) is mandatory, except for FULL mode.

app.invalid_num_threads=Invalid number of threads. Using default.

app.using_default_threads=Using default number of threads:

app.db_error=Database error:

app.file_error=File error:

app.general_error=General error:

app.execution_time=Execution time:

app.milliseconds=ms

app.usage_header=Usage: java -jar MegaSenaSuggester.jar [options]

app.param_n=-n <number_of_bets> Number of bets to generate (Integer > 0).

Mandatory unless -m FULL.

app.param_m=-m <mode> Generation mode (RANDOM, LEXICO, FULL, FILE).

Mandatory.

app.param_t=-t <num_threads> Number of threads (Integer > 0). Optional. Default is CPU cores.

app.param_d=-d <db_path> Path to the SQLite database. Optional. Default is MegaPower.db.

